

Poker AI Summer Project — Research-Grade Roadmap

Author: (you) **Mentor:** (your professor) **Duration:** 7 active weeks + 1 buffer week (May 12 → ~July 6, 2026) **Effort:** 7–8 hrs/week baseline, 10–12 hrs/week peak **Final deliverable:** End-June / first-week-July presentation + GitHub repo + short written report

0. Executive Summary

This project builds a poker bot through a principled ladder of strategies, where each rung is a strict generalization or improvement of the previous one. The ladder begins with random play, climbs through rule-based and equity-based heuristics, then enters the regret-minimization family (regret matching → vanilla CFR → CFR+ → outcome-sampling MCCFR → external-sampling MCCFR), and applies these to **Kuhn Poker** and **Leduc Hold'em**. A stretch goal of NFSP or Deep CFR on Leduc is included but explicitly optional.

Evaluation is research-grade: large-N duplicate matches, paired confidence intervals, head-to-head tournaments, **exploitability** computed against a best-response oracle, and convergence plots in log-log scale.

The project is intentionally narrow but deep: it will *not* attempt full Texas Hold'em. Texas Hold'em-scale work requires card abstraction, action abstraction, and either subgame solving or large-scale MCCFR — each of which is a multi-month effort on its own.

A. Recommended Environment

A.1 Framework comparison

Framework	Strength	Weakness	Suitability for this project
OpenSpiel (DeepMind)	Reference CFR / CFR+ / MCCFR / NFSP / Deep CFR; built-in <code>exploitability</code> and <code>best_response</code> solvers; Kuhn and Leduc as core games; consistent extensive-form API	C++ core with Python bindings; install can be finicky on some platforms; less RL-ergonomic	Primary. Game-theory-first design; everything you need is in the box.
RLCard	Pythonic; clean Gym-like API; built-in DQN/NFSP/DMC examples; supports Leduc + No-Limit Hold'em	CFR support is thinner; exploitability tooling is less mature; more RL-flavored than GT-flavored	Secondary. Useful as a sanity check and for NFSP if Deep CFR proves hard.

Framework	Strength	Weakness	Suitability for this project
PettingZoo	Standard multi-agent RL API; great ecosystem	No native CFR/MCCFR; you'd build evaluation infrastructure yourself	Not recommended as primary. Good for RL projects, not CFR research.
ACPC server / pokerkit	Industry-standard benchmark protocol	Heavyweight; designed for full Hold'em competition	Out of scope for 7 weeks.

A.2 Recommendation

Use OpenSpiel as the primary framework. Use RLCard only if you reach the NFSP stretch and OpenSpiel's NFSP gives you trouble.

Reasons specific to a *research* project:

1. OpenSpiel ships `pyspiel.exploitability(game, policy)` — without this you'd have to write a best-response solver yourself, which is a week of work.
2. Kuhn and Leduc are first-class games (`kuhn_poker`, `leduc_poker`), so you can validate your home-grown CFR against the library's `CFRSolver` line-by-line.
3. The same API exposes the *infostate tree*, which makes it possible to *visualize* learned strategies (e.g., bluff frequencies at each information set) — exactly the kind of qualitative analysis your professor will want.

A.3 Starting game

Game	Information sets	Why start here
Kuhn Poker	12	Closed-form Nash family (1-parameter); CFR converges in seconds; perfect for theory validation.
Leduc Hold'em	936	Still tractable for tabular CFR; introduces a public board card, two betting rounds, and a card-pairing rule — close to real poker structure.
Limit Texas Hold'em	$\sim 3 \times 10^{14}$	Requires card/action abstraction and weeks of compute. Out of scope.

Verdict: Kuhn first (Weeks 2-4), Leduc second (Weeks 5-6), and only experiment with simplified/abstracted Texas Hold'em as a Week 7+ stretch if all prior milestones are met.

B. Evaluation Design

B.1 Why "hands won" is the wrong metric

Poker payoffs are unequal across hands. A bot that wins 80% of small pots but loses one big all-in has a negative expected value despite a high win rate. The right unit is **money won per hand**, not hands won.

The standard reporting unit in research poker is:

$$\text{mbb/g} = \text{milli-big-blinds per game (hand)} = 1000 \cdot \frac{\text{net winnings in BB}}{\text{hands played}}$$

For Kuhn and Leduc, just use **chips per hand** (mean payoff per hand to player 1).

B.2 Right metrics

Metric	Definition	When to use
Mean payoff $\bar{u}_1$	Average chips won per hand by player 1	Every head-to-head match
95% CI on $\bar{u}_1$	$\bar{u}_1 \pm 1.96 \cdot s / \sqrt{N}$	Always report alongside mean
Win rate	Fraction of hands where $u_1 > 0$	Supplementary, not headline
Exploitability	$\varepsilon(\sigma) = \frac{1}{2} \left(\max_{\sigma'_1} u_1(\sigma'_1, \sigma_2) + \max_{\sigma'_2} u_2(\sigma_1, \sigma'_2) \right)$ in 2p0s games	Measures distance from Nash; the headline number for solver bots
NashConv	$2 \cdot \varepsilon$ in 2p0s games (OpenSpiel reports this)	Same thing, different convention
Best-response value	Payoff of the optimal counter-strategy	Diagnostic when exploitability seems off
Regret curve	R^T / T vs T on log-log axes	Convergence diagnostic for CFR

B.3 Variance reduction: duplicate evaluation

Poker has high payoff variance. Two techniques cut it dramatically:

1. **Duplicate (paired) evaluation.** For each random card deal, play two hands: one with bot A as P1 and bot B as P2, then swap. Average the two payoffs. Eliminates almost all card-luck variance.
2. **AIVAT** (Burch et al. 2018) is the gold standard but non-trivial to implement; skip unless time permits.

B.4 Evaluation protocol

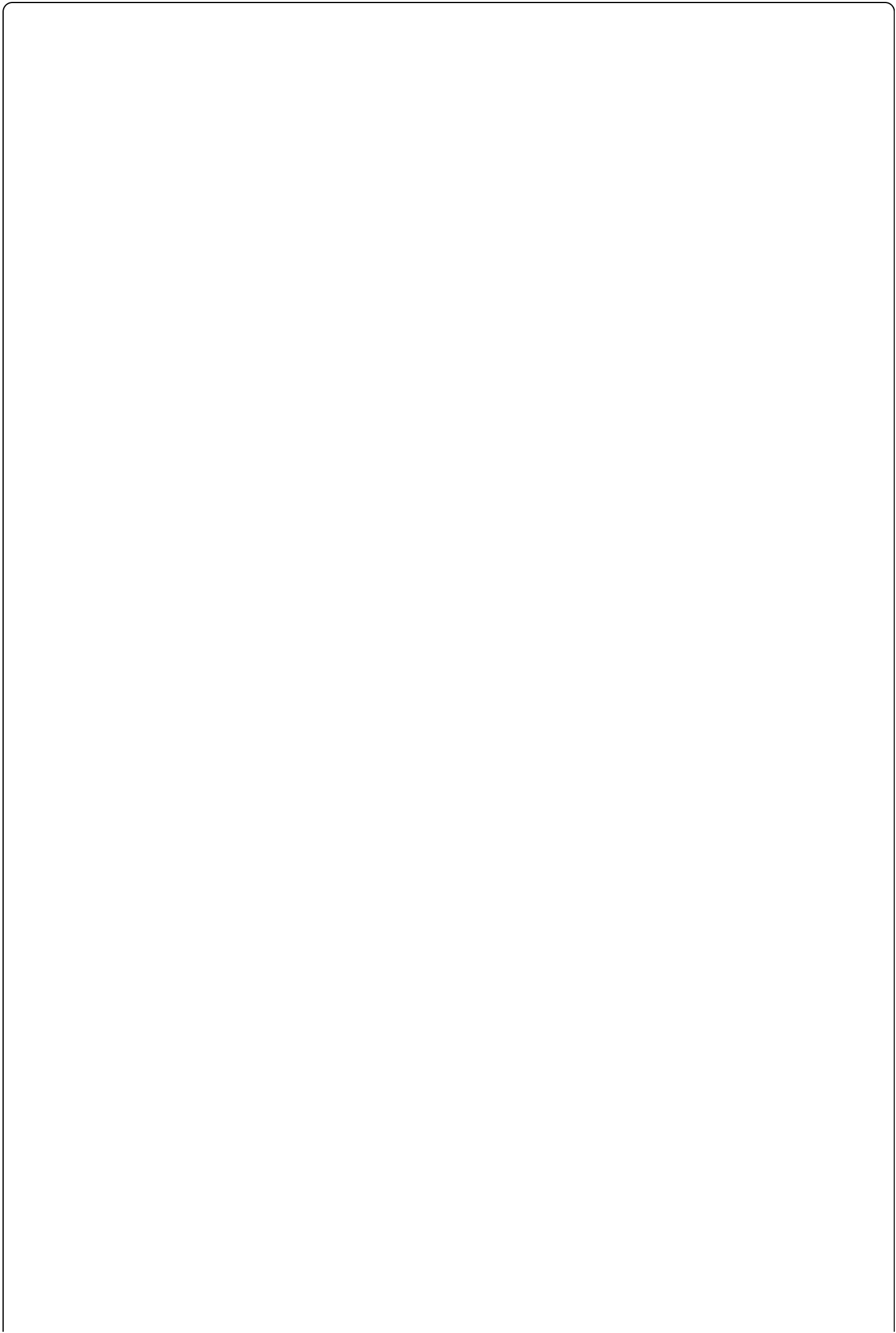
```
For each agent pair (A, B):  
    for seed in [0, 1, 2, 3, 4]:          # 5 independent runs  
        for hand in range(50_000):        # 50k duplicate hands = 100k total  
            deal cards with rng(seed, hand)  
            play (A=P1, B=P2) and (B=P1, A=P2) on same deal  
            record both payoffs  
Aggregate: mean  $\pm$  95% CI across the 5 seeds (cluster bootstrap is fine)
```

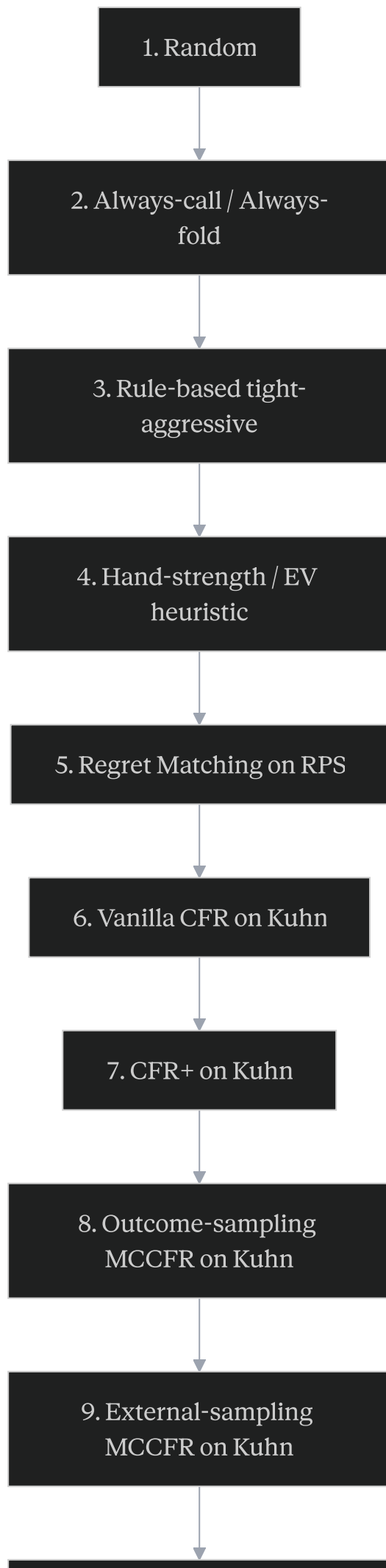
Hand counts. 50k duplicate hands gives $\sim 50k$ effective samples (since pairing removes most variance). For Leduc, payoff std-dev per hand is roughly 4 chips; CI half-width is then ~ 0.035 chips/hand at $N=50k$. That is publication-tight.

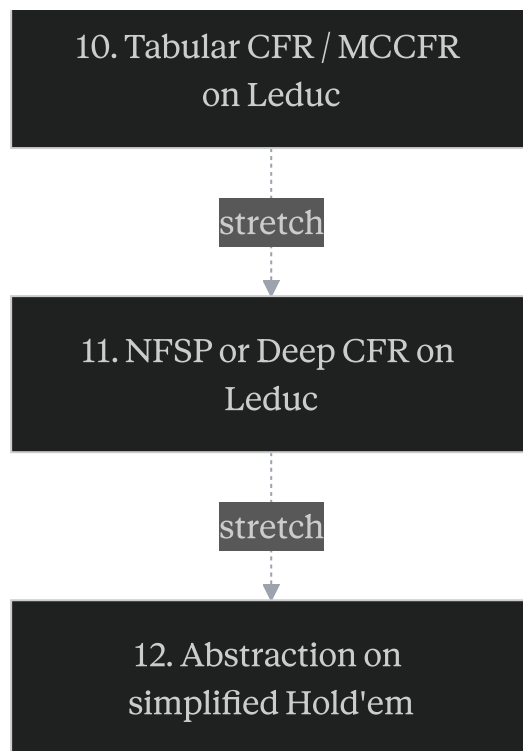
Reporting format. Every experiment row must include: game, agent A id+config, agent B id+config, N, seed list, mean payoff to A, 95% CI, win rate, p-value vs zero (one-sample t-test), date, git commit hash.

C. Bot / Strategy Ladder

Each rung either *strictly contains* the previous (e.g., $MCCFR \subset CFR$ class) or introduces a strictly new and necessary concept.







C.1 Detailed rungs

For each rung: *what / concept / math / paper / code / experiment / result*.

Rung 1 — Random agent

- *What*: Picks a legal action uniformly at random.
 - *Concept*: Baseline; defines the "floor" of skill.
 - *Math*: None.
 - *Paper*: —
 - *Code*: (~10 lines); one function `act(state) -> random.choice(state.legal_actions())`.
 - *Experiment*: 100k duplicate hands of Random vs Random on Kuhn. Sanity-check that mean payoff is statistically indistinguishable from 0 (it must be by symmetry).
 - **Result to show prof.** "Random vs Random gives $\bar{u}_1 = 0.001 \pm 0.012$ chips/hand on Kuhn, $N = 100000$, consistent with the symmetry argument."
-

Rung 2 — Always-call and Always-fold

- *What*: Two degenerate fixed strategies.
- *Concept*: Demonstrates that an exploitable fixed strategy can be *crushed* by a fitted counter-strategy, motivating Nash.
- *Math*: Trivial expected-value computation by hand on Kuhn.
- *Code*: (~10 lines) each.

- *Experiment:* Hand-compute the EV of Always-call vs uniform; verify empirically.
 - *Result:* Table of pairwise mean payoffs between {Random, Always-call, Always-fold}.
-

Rung 3 — Rule-based tight-aggressive bot

- *What:* Hand-coded rules using hole card + board card (in Leduc) and pot odds.
 - *Concept:* Heuristic exploitation; shows that domain knowledge beats blind play.
 - *Math:* Definitions of pot odds, equity. Pot odds = (call amount) / (pot + call amount); a call is +EV iff equity > pot odds.
 - *Code:* (~50 lines); if-then ladder on hand strength buckets.
 - *Experiment:* Tight-aggressive vs Random and vs Always-call on Kuhn and Leduc.
 - *Result:* Show that the heuristic bot beats Random by ~0.3 chips/hand on Kuhn but is itself beaten by later CFR bots — motivating equilibrium play.
-

Rung 4 — Hand-strength / expected-value bot

- *What:* For each decision, computes expected value of each action assuming opponent plays uniformly at random over consistent hands.
 - *Concept:* Expected utility maximization under a fixed opponent model. The first "principled" bot.
 - **Math:** $EV(a \mid I) = \sum_{h \sim I} P(h \mid I) \cdot \mathbb{E}[u \mid h, a]$, where h ranges over histories consistent with information set I .
 - *Code:* (~80 lines); enumeration is fine for Kuhn/Leduc.
 - *Experiment:* EV bot vs Rule-based and vs Random.
 - *Result:* EV bot beats Rule-based, but is *exploitable* — set up the punchline for Rung 5+.
-

Rung 5 — Regret matching on Rock-Paper-Scissors

- *What:* Implements the regret-matching update on a toy normal-form game.
- *Concept:* Introduces *regret*, *no-regret learning*, and the *folk theorem* that no-regret play in 2p0s games converges (in average strategies) to Nash.
- *Math:* (must understand)
 - External regret: $R^T(a) = \sum_{t=1}^T (u_t(a, \sigma_{-i}^t) - u_t(\sigma^t))$
 - Regret matching: $\sigma^{T+1}(a) \propto [R^T(a)]^+$ (positive part). If all regrets ≤ 0 , play uniformly.
 - Average strategy: $\bar{\sigma}^T(a) = \frac{1}{T} \sum_{t=1}^T \sigma^t(a)$
- *Paper:* Hart & Mas-Colell (2000), "A simple adaptive procedure leading to correlated equilibrium." Also Neller-Lanctot tutorial §2.

- **Code:** (~60 lines); the entire algorithm is 5 lines of NumPy inside a `for t in range(T)` loop.
 - ***Experiment:*** Self-play RPS for $T = 10^5$ iterations. Plot $\bar{\sigma}^T$ vs t .
 - ***Result:*** $\bar{\sigma}^T \rightarrow (1/3, 1/3, 1/3)$, and average regret $R^T/T \rightarrow 0$.
 - **Why this matters:** Every subsequent CFR variant is *regret matching applied locally at every information set*. Get this rung absolutely right.
-

Rung 6 — Vanilla CFR on Kuhn Poker

- **What:** Tabular counterfactual regret minimization on the full Kuhn extensive-form game.
 - **Concept:** Generalizes regret matching from normal-form to extensive-form by introducing *counterfactual* values.
 - **Math:** (must understand; see §D for full derivation)
 - Counterfactual reach: $\pi_{-i}^\sigma(h) = \prod_{(h',a) \prec h, P(h') \neq i} \sigma(h')(a)$
 - Counterfactual value: $v_i^\sigma(I) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \succeq h} \pi^\sigma(h, z) u_i(z)$
 - Immediate counterfactual regret at I for action a : $R^T(I, a) = \sum_t (v_i^{\sigma_{I \rightarrow a}^t}(I) - v_i^{\sigma^t}(I))$
 - Regret-matching update on each I : $\sigma^{T+1}(I)(a) \propto [R^T(I, a)]^+$
 - **Paper:** Zinkevich, Johanson, Bowling, Piccione (2007), "Regret Minimization in Games with Incomplete Information." Plus Neller-Lanctot tutorial.
 - **Code:** (~200 lines); recursive `cfr(state, reach_probs)` returning utility.
 - ***Experiment:*** Self-play CFR for $T = 10^4$ iterations. Plot exploitability vs T on log-log axes. Verify $\bar{\sigma}^T$ matches the known Kuhn Nash family σ_α parameterized by $\alpha \in [0, 1/3]$.
 - ***Result:*** Exploitability falls as $O(1/\sqrt{T})$; $\bar{\sigma}^T$ converges to a specific point in the Kuhn equilibrium family.
-

Rung 7 — CFR+ on Kuhn

- **What:** Replace regret matching with *regret matching plus*: clip cumulative regrets to be non-negative at every iteration, and use linear (weighted) averaging.
- **Concept:** Empirically converges roughly an order of magnitude faster, with stronger anytime guarantees.
- ***Math:*** Update becomes $R^{T+1}(I, a) = [R^T(I, a) + (v(I, a) - v(I))]^+$, and average strategy weights iteration t by t rather than uniformly.
- **Paper:** Tammelin (2014), "Solving Large Imperfect Information Games Using CFR+." Tammelin et al. (2015) Science paper on solving heads-up limit Hold'em.
- **Code:** (~50 lines) of diff from vanilla CFR.
- **Experiment:** Same as Rung 6, overlay CFR+ exploitability curve on top of CFR's.

- *Result:* CFR+ reaches the same exploitability $\sim 5\text{--}10\times$ faster.
-

Rung 8 — Outcome-sampling MCCFR on Kuhn

- *What:* Sample a single trajectory through the game tree per iteration; update regrets only along that trajectory using importance-sampled counterfactual values.
 - *Concept:* Trades variance for compute. Each iteration is $O(\text{depth})$ instead of $O(\text{tree size})$.
 - *Math:* (intuition + key formula; *skim* the variance bound proof)
 - On the sampled trajectory z , the unbiased estimator of $v_i^\sigma(I, a)$ uses $1/q(z)$ importance weights, where $q(z)$ is the sampling probability.
 - *Paper:* Lanctot, Waugh, Zinkevich, Bowling (2009), "Monte Carlo Sampling for Regret Minimization in Extensive Games." §3 (outcome sampling) is the critical section.
 - *Code:* (~150 lines).
 - *Experiment:* Plot exploitability vs *iterations* and vs *wall-clock time*. Show that wall-clock convergence of MCCFR beats vanilla CFR on larger games (preview of Leduc).
 - *Result:* Two side-by-side plots; demonstrate the variance-vs-compute tradeoff.
-

Rung 9 — External-sampling MCCFR on Kuhn

- *What:* Sample only the opponent's and chance's actions; iterate over all of *your* actions at each of your information sets.
 - *Concept:* Lower-variance than outcome sampling, but more compute per iteration. The most popular MCCFR variant in research.
 - **Math:** Same framework as Rung 8 with different sampling distribution q .
 - *Paper:* Lanctot et al. 2009 §4.
 - *Code:* (~150 lines).
 - *Experiment:* Add a third curve to the Rung 8 plots.
 - *Result:* External sampling sits between vanilla CFR and outcome sampling in per-iteration progress, and beats both in wall-clock on Leduc.
-

Rung 10 — Tabular CFR and external-sampling MCCFR on Leduc

- *What:* Apply Rungs 6–9 to Leduc Hold'em.
- *Concept:* Scaling test. Leduc has $\sim 80\times$ more information sets than Kuhn.
- *Math:* Identical to Kuhn rungs.
- *Code:* (~50 lines); mostly an env swap, but you may need to refactor your CFR to be game-agnostic.

- **Experiment:** Vanilla CFR vs CFR+ vs ES-MCCFR on Leduc. Run for $T = 10^6$ iterations. Plot exploitability vs wall-clock time. Compare your numbers against OpenSpiel's reference CFR — they should match to 3+ significant figures.
 - *Result:* Headline plot of the project: three convergence curves on the same axes, with OpenSpiel's CFR overlaid for validation.
-

Rung 11 — (Stretch) NFSP or Deep CFR on Leduc

- *What:* Replace tabular regret/strategy tables with neural networks.
 - *Concept:* Bridges CFR to deep RL; necessary for any game beyond Leduc.
 - *Math:* (intuition only) NFSP combines a Q-learner (best response) and a supervised learner (average strategy) via fictitious self-play. Deep CFR uses a regret network and a strategy network trained on MCCFR rollouts.
 - *Paper:* Heinrich & Silver (2016), "Deep Reinforcement Learning from Self-Play in Imperfect-Information Games" (NFSP); Brown, Lerer, Gross, Sandholm (2019), "Deep Counterfactual Regret Minimization."
 - *Code:* Use OpenSpiel's built-in `nfsp` or `deep_cfr` rather than rewriting.
 - *Experiment:* Train for ~1 hour; report exploitability vs your tabular ES-MCCFR.
 - *Result:* Tabular ES-MCCFR should beat NFSP on a small game like Leduc — the point of this rung is methodological (showing the scaling path), not winning.
-

Rung 12 — (Far stretch) Abstraction on simplified Hold'em

- Probably out of scope. Document this in "Future Work" instead.
-

D. Mathematical Concepts to Internalize

Group by week so you can pace explanation depth.

D.1 Foundations (Weeks 1-2)

Extensive-form game. A tuple $(N, H, P, \sigma_c, \mathcal{I}, u)$ where:

- N is the player set,
- H is the set of *histories* (sequences of actions),
- $P : H \rightarrow N \cup \{c\}$ tells whose turn it is at each non-terminal history,
- σ_c is the chance distribution at chance nodes,
- $\mathcal{I}_i$ partitions player i 's decision points into *information sets* (indistinguishable histories from i 's viewpoint),
- $u : Z \rightarrow \mathbb{R}^N$ gives utilities at terminal histories Z .

Information set. The central object of imperfect-information games. A player chooses an action knowing only the information set they are in, not the full history. *In Kuhn, after dealing and one bet, player 2 knows their own card and the action sequence, but not P1's card — that defines their info sets.*

Behavioral strategy. $\sigma_i(I)$ is a probability distribution over actions at info set I . Kuhn's theorem says behavioral strategies are equivalent to mixed strategies in games with perfect recall — so we use behavioral strategies because they're exponentially more compact.

D.2 Reach probabilities (Week 3)

Realization probability. $\pi^\sigma(h) = \prod_{(h',a) \preceq h} \sigma_{P(h')}(h')(a)$ — the joint probability of reaching h given all players (and chance) play according to σ .

****Counterfactual reach.**** $\pi_{-i}^\sigma(h)$ is the same product *excluding* player i 's own contribution. Intuition: "the probability we get to h assuming player i was **trying** to get there." This isolates the part of the reach probability that i can't control.

D.3 Counterfactual value (Week 3)

$$v_i^\sigma(I) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \in Z, z \succeq h} \pi^\sigma(h, z) u_i(z)$$

Intuition: weight each history in I by its counterfactual reach, then take expected utility under σ from there. The counterfactual weighting is what makes the regret update unbiased per info set.

D.4 Counterfactual regret and the CFR theorem (Week 3)

Immediate counterfactual regret. $R^T(I, a) = \sum_t (v_i^{\sigma_{I \rightarrow a}^t}(I) - v_i^{\sigma^t}(I))$, where $\sigma_{I \rightarrow a}^t$ is σ^t modified to deterministically take a at I .

****The key theorem (Zinkevich 2007).**** If at every information set we apply regret matching to immediate counterfactual regrets, then the **total** external regret of the resulting strategy is bounded by the sum of immediate counterfactual regrets, which is in turn $O(\sqrt{T})$ per info set. Consequently, in two-player zero-sum games with perfect recall, the **average** strategy profile is a 2ε -Nash equilibrium with $\varepsilon = O(1/\sqrt{T})$.

Why this is the "magic": you minimize regret locally and independently at each information set, and that suffices to drive the whole game's strategy to Nash. You do not need any global coordination.

You should be able to state this theorem and explain *why* (sum of local regrets bounds global regret + folk theorem). The proof itself can be cited rather than re-derived.

D.5 Exploitability (Weeks 3-6)

$$\varepsilon(\sigma) = \frac{1}{|N|} \sum_{i \in N} \left(\max_{\sigma'_i} u_i(\sigma'_i, \sigma_{-i}) - u_i(\sigma) \right)$$

For 2p0s games this simplifies to half the NashConv. **A Nash equilibrium has exploitability 0; any other strategy has positive exploitability.** This is your headline convergence metric.

D.6 MCCFR sampling (Weeks 4-5)

The sampling problem. Vanilla CFR traverses the full tree per iteration: $O(|\mathcal{I}| \cdot |A|)$ work. For Leduc that's fine; for anything larger it isn't.

Outcome sampling. Sample one terminal trajectory z from a sampling distribution q . Use importance weight $1/q(z)$ to keep the regret estimator unbiased. Variance is high; per-iteration cost is $O(\text{depth})$.

External sampling. Sample chance and opponent actions, enumerate own actions. Variance is much lower; cost is between outcome sampling and full CFR. *This is the workhorse for research-scale games.*

Variance vs computation. This is the central tradeoff; the bias is always zero (all variants give unbiased regret estimates), only variance and per-iteration cost change. (*Skim* the formal variance bounds in Lanctot 2009 §5; just internalize the qualitative ordering.)

D.7 Abstraction (mention only)

For Texas Hold'em scale, card abstraction (bucket private hands into $\sim 10^4$ groups) and action abstraction (restrict bet sizes) reduce the game enough that CFR variants become tractable. Mentioned in your future work slide; not implemented.

E. Curated Reading List

Listed in *the order you should read*, not in order of historical importance.

Bibliographic note. Cite the published version of each paper (Science / NeurIPS / IJCAI / AAAI) in your final report's references, not the arXiv preprint, where both exist.

F. Week-by-Week Roadmap

Assumes 7-8 hrs/week base, 10-12 hrs/week peak when noted. Each week ends with a 5-10 slide micro-deck for your professor.

Week 1 (May 12 - May 18) — Setup and baselines

- **Technical:** Install OpenSpiel; verify Kuhn and Leduc load; implement Random, Always-call, Always-fold, Rule-based, EV-heuristic bots.
- **Math:** Extensive-form games, information sets, behavioral strategies (§D.1).
- **Reading:** Neller-Lanctot tutorial §1-2; OpenSpiel "Getting Started" docs.
- **Coding:** Repo scaffold (see §G); `agents/random_agent.py`, `agents/rule_based.py`, `agents/ev_heuristic.py`; `evaluation/tournament.py`; `experiments/exp01_baseline_tournament.py`.
- **Experiment:** Round-robin among the five baselines on Kuhn. 100k duplicate hands per pair, 5 seeds.
- **Deliverable:** 5-slide deck — problem motivation; framework choice (1 slide); pairwise mean-payoff table with CIs (1 slide); next-week plan.
- **Risk / fallback:** OpenSpiel install fails on your platform → switch to RLCard for Kuhn; revisit OpenSpiel via Docker. **Budget at most one full session on install before falling back.**

Week 2 (May 19 - May 25) — Regret matching

- **Technical:** Implement regret matching from scratch on Rock-Paper-Scissors. Then on a 2-action Bayesian RPS (i.e., card-conditioned RPS) to bridge toward Kuhn.
- **Math:** External regret, regret matching update, average strategy, folk theorem for 2p0s (statement only) (§D.4 partial).
- **Reading:** Neller-Lanctot §2; Zinkevich 2007 §1-3 (slow read).
- **Coding:** `agents/regret_matching.py`, `experiments/exp02_rps_convergence.py`.
- **Experiment:** RPS self-play, $T = 10^5$. Plot (a) $\bar{\sigma}^T$ vs t for each action, (b) R^T/T vs t in log-log.
- **Deliverable:** 6-slide deck — what regret matching is; theorem statement; RPS convergence plot; preview of CFR.

- **Risk / fallback:** Math feels slow → spend an extra 2 hrs on Neller-Lanctot before pushing on.

Week 3 (May 26 - June 1) — Vanilla CFR on Kuhn ★ peak week

- **Technical:** Implement vanilla CFR on Kuhn, from scratch, then validate against OpenSpiel's `CFRSolver`.
- **Math:** Counterfactual reach, counterfactual value, counterfactual regret, the CFR theorem (§D.2-D.4).
- **Reading:** Zinkevich 2007 §3-4 carefully; Neller-Lanctot §3.
- **Coding:** `agents/cfr.py`, `evaluation/exploitability.py` (use OpenSpiel's), `experiments/exp03_kuhn_cfr_convergence.py`.
- **Experiment:** Self-play CFR for $T = 10^4$ on Kuhn. Plot exploitability vs T in log-log. Verify your $\bar{\sigma}^T$ identifies a known point in Kuhn's Nash family.
- **Deliverable:** 8-slide deck — CFR math; algorithm pseudocode; convergence plot; Kuhn Nash family + recovered strategy table.
- **Risk / fallback:** CFR doesn't converge or numbers don't match OpenSpiel → strict diff against OpenSpiel's reference; the bug is almost always in counterfactual reach or in the recursion's pass-through of reach probabilities. Allocate up to 12 hrs this week.

Week 4 (June 2 - June 8) — CFR+ and outcome-sampling MCCFR

- **Technical:** Implement CFR+; implement outcome-sampling MCCFR.
- **Math:** Linear averaging in CFR+; importance sampling for MCCFR; variance vs computation (§D.6).
- **Reading:** Tammelin 2014 note; Lanctot 2009 §1-3.
- **Coding:** `agents/cfr_plus.py`, `agents/mccfr_outcome.py`, `experiments/exp04_cfr_variants_kuhn.py`.
- **Experiment:** Overlay convergence curves on Kuhn for CFR, CFR+, and outcome-sampling MCCFR — both per-iteration and per-second.
- **Deliverable:** 7-slide deck — CFR+ update rule; OS-MCCFR derivation; the three-curve plot; takeaway on tradeoff.
- **Risk / fallback:** OS-MCCFR has high variance and may look noisy at 10^4 — run longer (10^5 – 10^6) and average across seeds.

Week 5 (June 9 - June 15) — External-sampling MCCFR and move to Leduc

- **Technical:** Implement external-sampling MCCFR; refactor agents to be game-agnostic; bring up Leduc.
- **Math:** External sampling distribution; perfect recall as a precondition (cite).
- **Reading:** Lanctot 2009 §4; OpenSpiel Leduc docs.
- **Coding:** `agents/mccfr_external.py`; refactor `agents/cfr.py` to accept any `pyspiel.Game`; `experiments/exp05_leduc_solvers.py`.

- **Experiment:** Vanilla CFR, CFR+, OS-MCCFR, ES-MCCFR on Leduc, T up to 10^6 . Plot exploitability vs wall-clock. Overlay OpenSpiel's reference CFR for validation.
- **Deliverable:** 8-slide deck — ES vs OS; refactor diagram; the headline Leduc convergence plot.
- **Risk / fallback:** Leduc CFR is much slower $\rightarrow$ cap T at 10^5 for vanilla and let MCCFR variants run longer; the *qualitative* ranking is what matters.

Week 6 (June 16 - June 22) — Head-to-head evaluation suite + stretch

- **Technical:** Build the full evaluation suite: round-robin tournaments, paired CIs, regret curves, strategy visualizations (bluff/call frequencies at each info set). Stretch: NFSP via OpenSpiel.
- **Math:** Best-response computation (statement); strategy visualization (no new math).
- **Reading:** OpenSpiel `(nfsp)` example; Heinrich-Silver 2016 §2 + §4 if attempting NFSP.
- **Coding:** `(evaluation/tournament.py)` (extend), `(evaluation/visualize_strategy.py)`, optional `(agents/nfsp_openspiel.py)` wrapper.
- **Experiment:** Round-robin among {Random, Rule-based, EV, CFR, CFR+, OS-MCCFR, ES-MCCFR, (NFSP)} on Kuhn and Leduc. Generate the strategy heatmap for the CFR+ Leduc solution.
- **Deliverable:** 10-slide deck — tournament table; convergence summary; bluff-frequency visualization; NFSP comparison if attempted.
- **Risk / fallback:** NFSP doesn't fit in the week $\rightarrow$ drop it; the tabular results are the project's substance.

Week 7 (June 23 - June 29) — Write-up, final plots, presentation

- **Technical:** Lock all code; freeze a `(v1.0)` tag; regenerate every plot from seeded scripts; write `(reports/final.md)` (~6 pages).
- **Math:** No new content; polish how you explain the CFR theorem.
- **Reading:** Skim Brown-Sandholm Libratus for the final motivation slide.
- **Coding:** `(scripts/regenerate_all_figures.py)`; ensure every figure in the report and slides traces to a script + seed.
- **Experiment:** None new; reproduce the final headline figures from fresh runs to confirm seed-determinism.
- **Deliverable:** Final presentation (~25 slides, §I structure) + written report + tagged repo.
- **Risk / fallback:** Always keep this week protected. If Week 6 slips, drop NFSP, not the polish.

Week 8 (June 30 - July 6) — Buffer / stretch / Deep CFR

- Use only if Week 7 was clean. Try Deep CFR via OpenSpiel; otherwise, polish the GitHub README so the project is publicly presentable.
-

G. Repository Structure

```
poker-ai/
├─ README.md                # Project overview, install, how to reproduce
├─ pyproject.toml           # Or requirements.txt; pin OpenSpiel, NumPy,
Matplotlib, pandas
├─ LICENSE
├─ configs/
|   ├─ kuhn_cfr.yaml        # Hyperparams: T, log_every, seed
|   ├─ kuhn_cfr_plus.yaml
|   ├─ kuhn_mccfr_outcome.yaml
|   ├─ leduc_mccfr_external.yaml
|   └─ leduc_nfsp.yaml     # (stretch)
├─ poker_ai/
|   ├─ __init__.py
|   └─ agents/
|       └─ base.py         # Abstract Agent class with act(info_state) ->
action
|           └─ random_agent.py
|           └─ always_call.py
|           └─ always_fold.py
|           └─ rule_based.py
|           └─ ev_heuristic.py
|           └─ regret_matching.py # For RPS / normal-form
|           └─ cfr.py           # Vanilla CFR
|           └─ cfr_plus.py
|           └─ mccfr_outcome.py
|           └─ mccfr_external.py
|           └─ nfsp_openspiel.py # (stretch) thin wrapper
├─ env/
|   └─ kuhn.py              # Mostly thin wrapper around pyspiel
|   └─ leduc.py
|   └─ rps.py              # Normal-form RPS for Rung 5
├─ evaluation/
|   └─ tournament.py        # Round-robin runner with duplicate pairing
|   └─ exploitability.py    # Wraps pyspiel.exploitability
|   └─ stats.py             # CIs, paired t-tests, bootstrap
|   └─ visualize_strategy.py # Info-set strategy heatmaps
├─ utils/
|   └─ logging.py           # Structured JSON + CSV logs
|   └─ seeding.py
|   └─ plotting.py         # Matplotlib defaults (dark theme!)
├─ experiments/
|   └─ exp01_baseline_tournament.py
|   └─ exp02_rps_convergence.py
|   └─ exp03_kuhn_cfr_convergence.py
```

```

|   ├── exp04_cfr_variants_kuhn.py
|   ├── exp05_leduc_solvers.py
|   ├── exp06_final_tournament.py
|   └── exp07_nfsp_leduc.py          # (stretch)
├── scripts/
|   ├── regenerate_all_figures.py
|   └── validate_against_openspiel.py
├── notebooks/
|   ├── 01_explore_kuhn_tree.ipynb
|   ├── 02_inspect_cfr_strategy.ipynb
|   └── 03_strategy_visualizations.ipynb
├── results/
|   ├── logs/                      # Per-experiment JSONL logs
|   ├── tables/                   # CSVs for the reporting table format
|   ├── figures/                  # PNG and PDF outputs
|   └── checkpoints/              # Saved strategy tables
├── reports/
|   ├── weekly/
|   |   ├── week01.md
|   |   ├── week02.md
|   |   └── ...
|   └── final.md                  # ~6-page report
├── slides/
|   ├── weekly/
|   └── final.pdf
└── tests/
    ├── test_cfr_kuhn_matches_openspiel.py
    ├── test_duplicate_pairing.py
    └── test_regret_matching_rps.py

```

Module responsibilities (terse):

- `agents/base.py`: defines the interface every agent must implement; downstream code never special-cases an agent type.
 - `evaluation/tournament.py`: the **only** place hands are played; every experiment delegates to it.
 - `evaluation/exploitability.py`: wraps OpenSpiel; provides a uniform Python API so you can swap in your own best-responder later.
 - `utils/seeding.py`: every script must call `seed_everything(seed)` first; reproducibility is non-negotiable.
 - `tests/`: at least one test per agent that validates against a known invariant (e.g., regret matching on RPS converges to uniform; CFR on Kuhn matches OpenSpiel).
-

H. Experiment Tracking

H.1 Logging convention

Every experiment writes a single newline-delimited JSON log to

`results/logs/{exp_id}_{timestamp}_{seed}.jsonl`. Each line is one logged step (typically every `log_every` CFR iterations):

```
json
{
  "exp_id": "exp03_kuhn_cfr_convergence",
  "git_commit": "a1b2c3d",
  "timestamp": "2026-05-27T14:31:02Z",
  "game": "kuhn_poker",
  "agent": "cfr_vanilla",
  "config": {"T": 10000, "seed": 0, "log_every": 100},
  "iteration": 1000,
  "wall_clock_s": 4.21,
  "exploitability": 0.0312,
  "nash_conv": 0.0625,
  "avg_strategy_snapshot_path": "results/checkpoints/exp03_seed0_iter1000.npz"
}
```

Aggregate logs into CSV via `scripts/aggregate_logs.py`. Plots are produced from CSVs, never from raw logs, so figure generation is fast and deterministic.

H.2 Results table format

Final reporting table (one row per matchup):

Game	Agent A	Agent B	N (duplicate hands)	Seeds	$\bar{u}_A$	95% CI	p-value (vs 0)	Exploitability(A)	Exploit
Kuhn	CFR ($T = 10^4$)	Random	50000	0-4	+0.198	[+0.191, +0.205]	<1e-10	0.0031	0.5208
Kuhn	CFR+ ($T = 10^4$)	CFR ($T = 10^4$)	50000	0-4	+0.002	[-0.005, +0.009]	0.58	0.0008	0.0031
Leduc	ES-MCCFR ($T = 10^6$)	Rule-based	50000	0-4	+1.45	[+1.38, +1.52]	<1e-10	0.012	1.93

This is your final-slide "results table" — populate every cell.

H.3 What gets versioned

- All code, configs, scripts.
 - All seeds (in the configs).
 - All produced CSVs and PNGs **for the final report only**; intermediate logs are git-ignored.
- `scripts/regenerate_all_figures.py` is your reproducibility contract.

I. Final Presentation Structure (~25 slides)

1. **Title** — name, advisor, date.
2. **Motivation** — why poker; Libratus / Pluribus headline; what's still hard (1 slide each, 2 total).
3. **Imperfect-information games** — extensive form, info sets, contrast with chess.
4. **Why "win rate" isn't the right metric** — payoff variance, equivalence to mbb/g, exploitability.
5. **Baseline ladder** — Random → Always-call → Rule-based → EV.
6. **Baseline results table.**
7. **Regret and no-regret learning** — the RPS experiment.
8. **Counterfactual values and counterfactual regret** — the core idea, with a diagram.
9. **CFR theorem statement** — and the intuition that local regret minimization suffices.
10. **Vanilla CFR on Kuhn** — algorithm + convergence plot + recovered Nash strategy.

11. **CFR+** — update rule + comparison plot.
12. **MCCFR variants** — outcome vs external sampling, with the variance/computation tradeoff.
13. **Scaling to Leduc** — info-set count, why MCCFR matters.
14. **Leduc convergence (headline plot).**
15. **Strategy visualization** — bluff frequency heatmap on Leduc.
16. **Head-to-head tournament results table.**
17. **Validation against OpenSpiel reference.**
18. *(if attempted)* **NFSP / Deep CFR comparison.**
19. **Failure cases and lessons** — be candid: variance issues, debugging stories.
20. **Reproducibility** — repo, seeds, single-command regeneration.
21. **Future work** — abstraction, full Hold'em, neural CFR variants.
22. **Limitations** — small games, no multi-player, no opponent modeling.
23. **References** (2 slides).
24. **Acknowledgments** + Q&A.

Tip: put one *concrete number* on every results slide. "CFR+ reached exploitability 0.001 on Leduc in 12 minutes" lands; "CFR+ converged faster" doesn't.

J. Implementation Details

J.1 Environment setup

```
bash

# Python 3.10 or 3.11 recommended (OpenSpiel sometimes lags newest Python)
python -m venv .venv
source .venv/bin/activate

pip install --upgrade pip
pip install open_spiel numpy matplotlib pandas pyyaml tqdm scipy
pip install jupyter ipykernel          # for notebooks
pip install pytest                     # for tests

# Quick smoke test:
python -c "import pyspiel; g = pyspiel.load_game('kuhn_poker'); print(g.num_distinct)"
# Expected: 2 4
```

If OpenSpiel install misbehaves on your machine, fall back to a Docker image:

```
bash
```

```
docker run -it --rm -v "$PWD":/workspace -w /workspace python:3.11-slim bash
# then pip install as above
```

J.2 Validate-against-OpenSpiel pattern (use this from Week 3 onward)

```
python

# scripts/validate_against_openspiel.py
import pyspiel
from open_spiel.python.algorithms import cfr, exploitability
from poker_ai.agents.cfr import VanillaCFR # your implementation

game = pyspiel.load_game("kuhn_poker")
T = 1000

# OpenSpiel reference
ref = cfr.CFRSolver(game)
for _ in range(T):
    ref.evaluate_and_update_policy()
ref_avg = ref.average_policy()
ref_expl = exploitability.exploitability(game, ref_avg)

# Your implementation
mine = VanillaCFR(game)
for _ in range(T):
    mine.step()
mine_avg = mine.average_policy()
mine_expl = exploitability.exploitability(game, mine_avg)

print(f"OpenSpiel exploitability: {ref_expl:.6f}")
print(f"Yours exploitability:      {mine_expl:.6f}")
# They should agree to 4+ decimals at T=1000.
```

J.3 Minimal vanilla CFR skeleton (Kuhn)

```
python
```

```

# poker_ai/agents/cfr.py (sketch – not full implementation)
import numpy as np
from collections import defaultdict

class VanillaCFR:
    def __init__(self, game):
        self.game = game
        self.regret_sum = defaultdict(lambda: np.zeros(game.num_distinct_actions()))
        self.strategy_sum = defaultdict(lambda: np.zeros(game.num_distinct_actions()))
        self.iter = 0

    def _current_strategy(self, info_key, legal_mask):
        r_plus = np.maximum(self.regret_sum[info_key], 0) * legal_mask
        s = r_plus.sum()
        if s > 0:
            return r_plus / s
        # uniform over legal
        u = legal_mask / legal_mask.sum()
        return u

    def _cfr(self, state, reach_probs):
        # reach_probs[i] =  $\pi_i^{\sigma}$  reach to current state for each player
        if state.is_terminal():
            return np.array(state.returns())
        if state.is_chance_node():
            ev = np.zeros(self.game.num_players())
            for a, p in state.chance_outcomes():
                rp = reach_probs.copy()
                # chance contributes to all players' "opponent" reach
                child = state.child(a)
                ev += p * self._cfr(child, rp * p_for_chance(rp, p))
            return ev
        player = state.current_player()
        info_key = state.information_state_string(player)
        legal_mask = np.zeros(self.game.num_distinct_actions())
        for a in state.legal_actions():
            legal_mask[a] = 1.0
        sigma = self._current_strategy(info_key, legal_mask)
        action_utils = {}
        node_util = np.zeros(self.game.num_players())
        for a in state.legal_actions():
            new_reach = reach_probs.copy()
            new_reach[player] *= sigma[a]
            u = self._cfr(state.child(a), new_reach)
            action_utils[a] = u
            node_util += sigma[a] * u

```

```

# Update regrets and strategy sum (only for the acting player)
opp_reach = np.prod(np.delete(reach_probs, player))
own_reach = reach_probs[player]
for a in state.legal_actions():
    regret = action_utils[a][player] - node_util[player]
    self.regret_sum[info_key][a] += opp_reach * regret
    self.strategy_sum[info_key][a] += own_reach * sigma[a]
return node_util

def step(self):
    root = self.game.new_initial_state()
    self._cfr(root, np.ones(self.game.num_players()))
    self.iter += 1

def average_policy(self):
    # Build an OpenSpiel TabularPolicy for plug-in exploitability.
    # Implementation left as exercise: normalize each info set's strategy_sum.
    ...

```

This is intentionally a *sketch*. Treat the actual implementation as the central deliverable of Week 3. The Neller–Lanctot tutorial gives a Python implementation that almost lines up with this skeleton; you should be able to map between them.

J.4 Exploitability evaluation

```

python

# poker_ai/evaluation/exploitability.py
from open_spiel.python.algorithms import exploitability as os_exp

def compute_exploitability(game, tabular_policy):
    """Returns exploitability (= NashConv / 2 in 2p0s games)."""
    return os_exp.exploitability(game, tabular_policy)

```

J.5 Duplicate-pair tournament loop

```

python

```

```

# poker_ai/evaluation/tournament.py (sketch)
import numpy as np
import pyspiel

def play_one_hand(game, agents, deal_seed):
    rng = np.random.default_rng(deal_seed)
    state = game.new_initial_state()
    while not state.is_terminal():
        if state.is_chance_node():
            outcomes = state.chance_outcomes()
            actions, probs = zip(*outcomes)
            a = rng.choice(actions, p=probs)
            state.apply_action(int(a))
        else:
            p = state.current_player()
            a = agents[p].act(state)
            state.apply_action(a)
    return state.returns() # length-2 array of utilities

def duplicate_match(game, agent_a, agent_b, n_pairs, base_seed):
    payoffs_a = []
    for k in range(n_pairs):
        # Same deal, swapped seats
        u1 = play_one_hand(game, [agent_a, agent_b], deal_seed=base_seed + k)
        u2 = play_one_hand(game, [agent_b, agent_a], deal_seed=base_seed + k)
        payoffs_a.append(0.5 * (u1[0] + u2[1]))
    return np.array(payoffs_a)

```

J.6 OpenSpiel quickstart for Kuhn / Leduc

python

```

import pyspiel
from open_spiel.python.algorithms import cfr, exploitability

# Kuhn
g = pyspiel.load_game("kuhn_poker")
solver = cfr.CFRSolver(g)
for it in range(10_000):
    solver.evaluate_and_update_policy()
    if it % 100 == 0:
        eps = exploitability.exploitability(g, solver.average_policy())
        print(f"iter={it:6d}   exploitability={eps:.6f}")

# Leduc – exactly the same pattern
g = pyspiel.load_game("leduc_poker")
# ... etc.

```

This is the reference you check your own CFR against. **Both should produce essentially identical exploitability curves.**

K. Minimum Successful Project vs Stretch

K.1 Minimum successful project (MSP)

Achievable in 7 weeks at 7-8 hrs/week even under setbacks:

- Working OpenSpiel pipeline.
- Five baseline agents (Random, Always-call, Always-fold, Rule-based, EV-heuristic).
- Regret matching on RPS, with convergence plot.
- Vanilla CFR on Kuhn, validated against OpenSpiel, with exploitability curve.
- CFR+ on Kuhn.
- Outcome-sampling and external-sampling MCCFR on Kuhn.
- ES-MCCFR on Leduc (convergence curve to exploitability < 0.05).
- Tournament table with paired CIs across all the above.
- Final presentation + 6-page report + clean, reproducible repo.

K.2 Stretch goals (only if MSP locks in by end of Week 6)

1. CFR+ on Leduc to exploitability < 0.005 .
2. NFSP on Leduc via OpenSpiel; compare exploitability vs tabular ES-MCCFR.
3. Deep CFR on Leduc.
4. Strategy visualization: bluff frequencies on Leduc as a heatmap over information sets.
5. AIVAT-style variance reduction in the evaluator.

K.3 What is explicitly *not* in scope

- Full Texas Hold'em (Limit or No-Limit).
- Card or action abstraction.
- Subgame solving, depth-limited search, real-time refinement.
- Multi-player (3+) poker.
- Opponent modeling.

Mention these in the "Future Work" slide; don't promise them.

Appendix: Single-glance project Gantt

Week	Mon dates	Theme	Math focus	Coding focus	Plot deliverable
1	May 12	Setup + baselines	EFG, info sets	Repo, 5 agents, tournament	Baseline pairwise table
2	May 19	Regret matching	External regret, RM update	RPS regret- matching	RPS convergence plot
3	May 26	Vanilla CFR on Kuhn ★	CF reach, CF value, CFR thm	CFR from scratch	Kuhn exploitability log-log
4	Jun 2	CFR+ and OS- MCCFR	Linear avg, importance sampling	Two new solvers	Three-curve Kuhn plot
5	Jun 9	ES-MCCFR and Leduc	Variance/compute tradeoff	Game-agnostic refactor + Leduc	Leduc convergence (headline)
6	Jun 16	Eval suite + stretch	Best response, exploitability	Full tournament, viz	Tournament table + heatmap
7	Jun 23	Final write-up	Polish only	Freeze, regenerate	Final report + slides
8	Jun 30	Buffer	—	—	Optional Deep CFR plot

End of roadmap. Treat this as v0.1 — revise after Week 1 once you've calibrated your actual pace against the plan.